Computer Olympic POSN KU Center

Topic : Binary Search Tree

Tanaboon Tongbuasirilai
Department of Computer Science
Kasetsart University

Agenda

- Introduction
- Properties
- Applications
- Traversals
- Operations

Reference

- https://www.geeksforgeeks.org/introduction-to-binary-search-tree-data-structure-and-algorithm-tutorials
- https://www.scholarhat.com/tutorial/datastructures/binary-search-tree-in-data-structures
- https://www.cs.cornell.edu/courses/cs4120/2016sp/lectures/lec_heaps/
- https://users.cs.duke.edu/~reif/courses/alglectures/skiena.lectures/lecture8.pdf

Introduction to Binary Search Tree (BST)

- A Binary Search Tree (BST) is a special type of binary tree in which the left child of a node has a value less than the node's value and the right child has a value greater than the node's value.

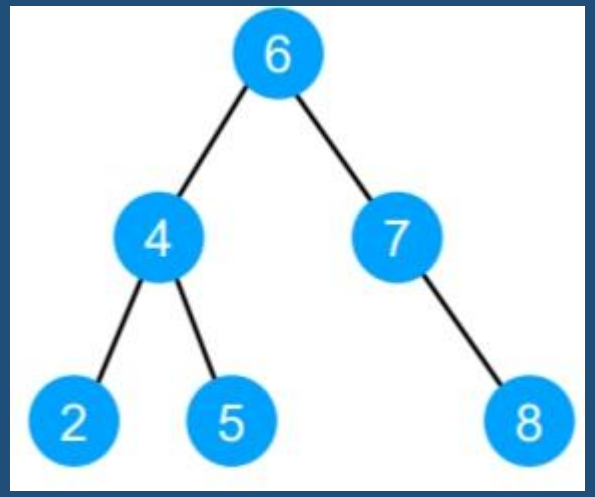
- This property is called the BST property, and it makes it possible to efficiently search, insert, and delete elements in the tree.

- The root of a BST is the node that has the smallest value in the left subtree and the largest value in the right subtree. Each left subtree is a BST with nodes that have smaller values than the root and each right subtree is a BST with nodes that have larger values than the root.

Mathematical definition

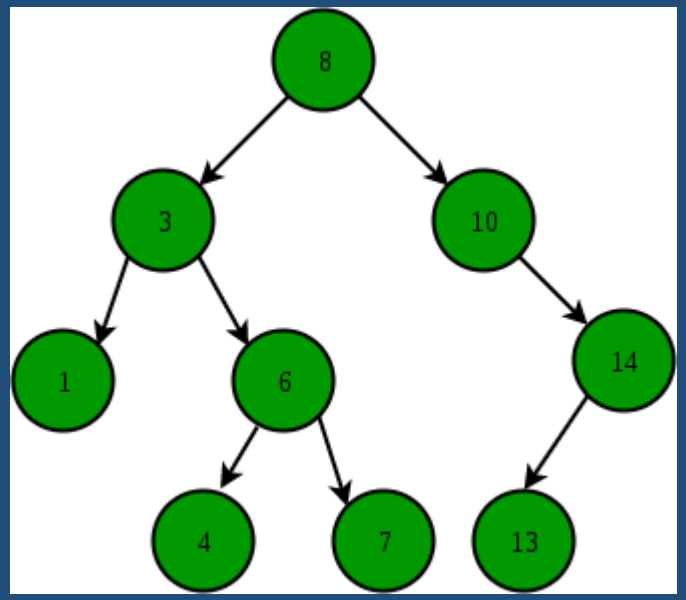
- Let S be a set whose elements have an order relation. For example, S could be a set of integers. A binary search tree for S is a proper binary tree T such that:
  - Each node x of T stores an element of S, denoted with x.key.
  - For each internal node x of T, the elements stored in the left subtree of x are less than or equal to x.key and the elements stored in the right subtree of x are greater than to x.key

- The keys (elements) in a binary search tree are always stored in such a way as to satisfy the binary-search-tree property:
  - "Let x be a node in a binary search tree. If y is a node in the left subtree of x, then y.key ≤ x.key. If y is a node in the right subtree of x, then y.key > x.key."



BST properties

- The left subtree of a node contains only nodes with keys lesser than the node's key.
- The right subtree of a node contains only nodes with keys greater than the node's key.
- This means everything to the left of the root is less than the value of the root and everything to the right of the root is greater than the value of the root. Due to this performing, a binary search is very easy.
- The left and right subtree each must also be a binary search tree.
- There must be no duplicate nodes(BST may have duplicate values with different handling approaches).

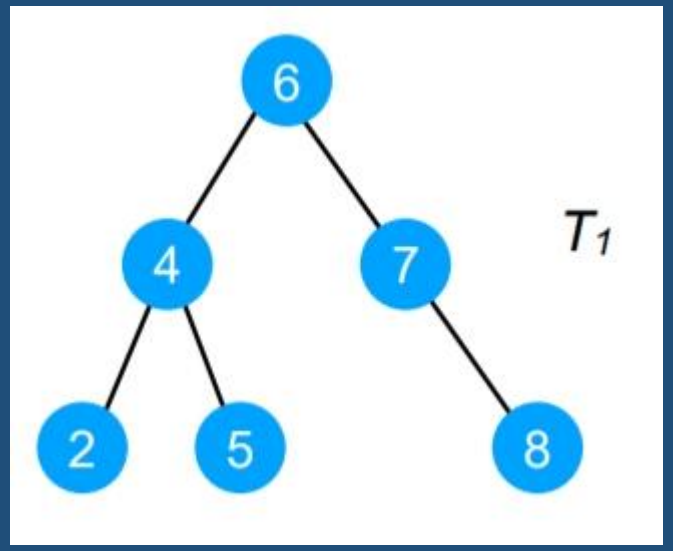


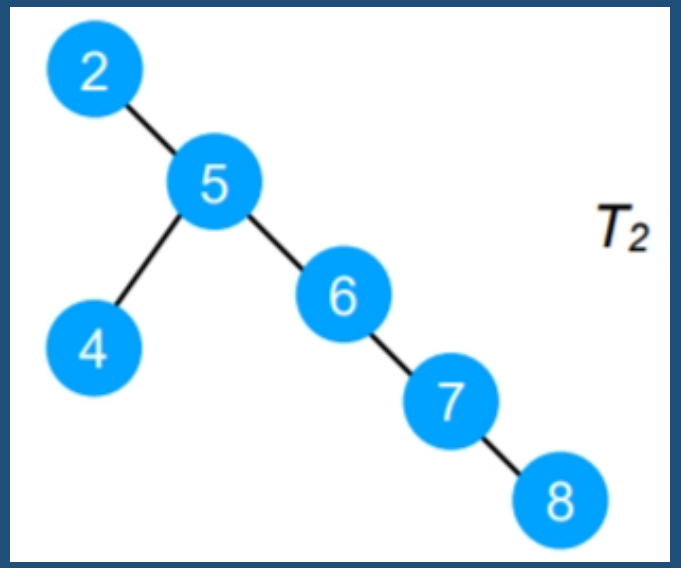
Applications

- **Databases**: Many databases use BSTs to store and search for data. For example, MySQL and SQLite use BSTs to store indexes of the data they contain, making queries much faster.
- **File Systems**: File systems use BSTs to store and organize files on a hard drive. For example, the Windows NTFS file system uses BSTs to store the Master File Table (MFT), which contains information about all the files on the hard drive.
- **Sorting**: BSTs can be used as a sorting algorithm. By inserting elements in a BST and then traversing it in-order, the elements can be sorted in O(nlog n) time.
- **Machine Learning**: BSTs can be used in decision tree-based machine learning algorithms, where the decision tree is a binary search tree that makes decisions based on the input features.
- **Priority Queue**: One simple implementation of a priority queue is as a binary search tree, using element priorities as keys. New elements are added by using the ordinary BST add operation. The minimum element in the tree can be found by simply walking leftward in the tree as far as possible and then pruning or splicing out the element found there.

Balanced vs Unbalanced BST

- The worst-case running time for most search-tree operations is proportional to the height of the tree.
- T1 is a binary search tree on 6 nodes with height 2.
- T2 is a less efficient binary search tree with height 4 that contains the same keys. (Unbalanced)
- Self-Balancing Binary Search Trees are height-balanced binary search trees that automatically keep the height as small as possible when insertion and deletion operations are performed on the tree.





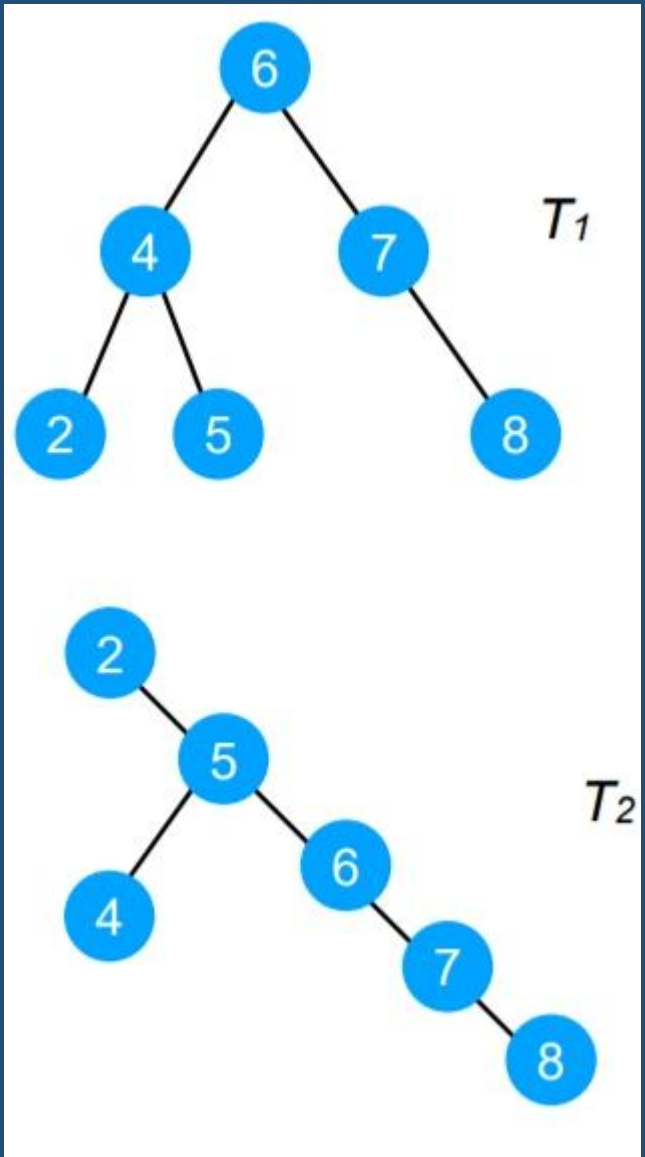
BST traversals

- Preorder traversal:
  - T1: 6, 4, 2, 5, 7, 8
  - T2: 2, 5, 4, 6, 7, 8

- Postorder traversal:
  - T1: 2, 5, 4, 8, 7, 6
  - T2: 4, 8, 7, 6, 5, 2

- **Inorder traversal:
  - T1: 2, 4, 5, 6, 7, 8
  - T2: 2, 4, 5, 6, 7, 8

- Remark: According to the binary-search-tree property, an inorder traversal of a binary search tree T visits the elements in sorted order



BST common operations

- Insert
- Search
- Minimum
- Maximum
- Successor
- Predecessor

Coding

```c
struct node
{
    int key;
    struct node* parent;
    struct node* left;
    struct node* right;
};
```

```c
struct node* createNode(int key = 0, struct node* parent = NULL)
{
    // Allocate memory for new node
    struct node* node = (struct node*)malloc(sizeof(struct node));
    // Assign key to this node
    node->key = key;
    // Initialize parent
    node->parent = parent;
    // Initialize left child, and right sibling as NULL
    node->left = NULL;
    node->right = NULL;
    return node;
}
```

Insertion

```c
// A utility function to insert
// a new node with given key in BST
struct node* insert(struct node* node, int key, struct node* parent = NULL)
{
    // If the tree is empty, return a new node
    if (node == NULL)
        return createNode(key, parent);

    // Otherwise, recur down the tree
    if (key < node->key)
        node->left = insert(node->left, key, node);
    else if (key > node->key)
        node->right = insert(node->right, key, node);

    // Return the (unchanged) node pointer
    return node;
}
```

```c
int main()
{
    struct node* node5 = createNode(5);
    struct node* node2 = insert(node5, 2);
    struct node* node6 = insert(node5, 6);
    insert(node5, -1);
    insert(node5, 3);
    /*
          5
         / \
        2   6
       / \
     -1   3
    */
```

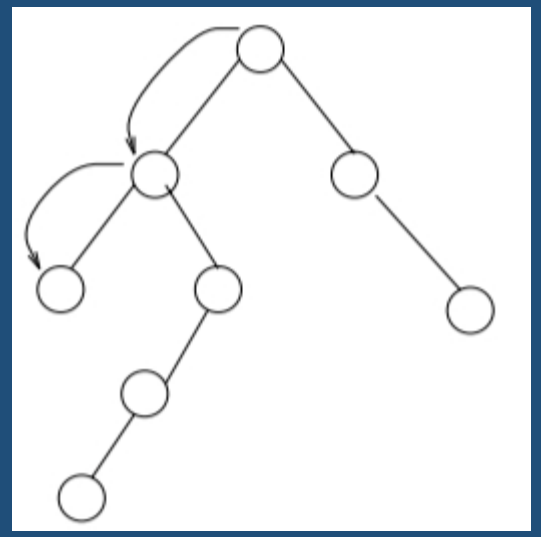
Search

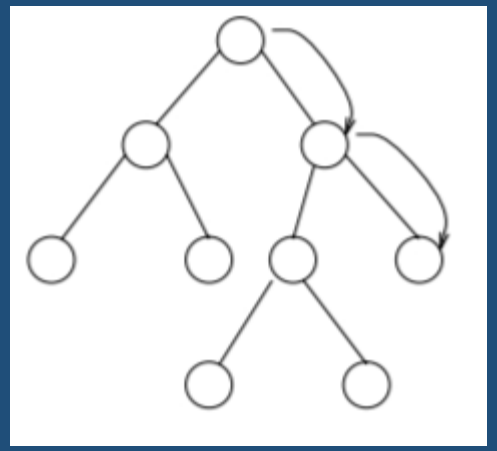
```c
struct node* search(int key, struct node* node)
{
    if ((node == NULL) || (key == node->key))
        return node;
    if (key < node->key)
        return search(key, node->left);
    else
        return search(key, node->right);
}
```

```c
/*
        5
       / \
      2   6
     / \
   -1   3
*/

printf("\n %d", search(6, node5)->key);
if (search(4, node5) == NULL)
    printf("\nnot found");
```

Minimum, Maximum

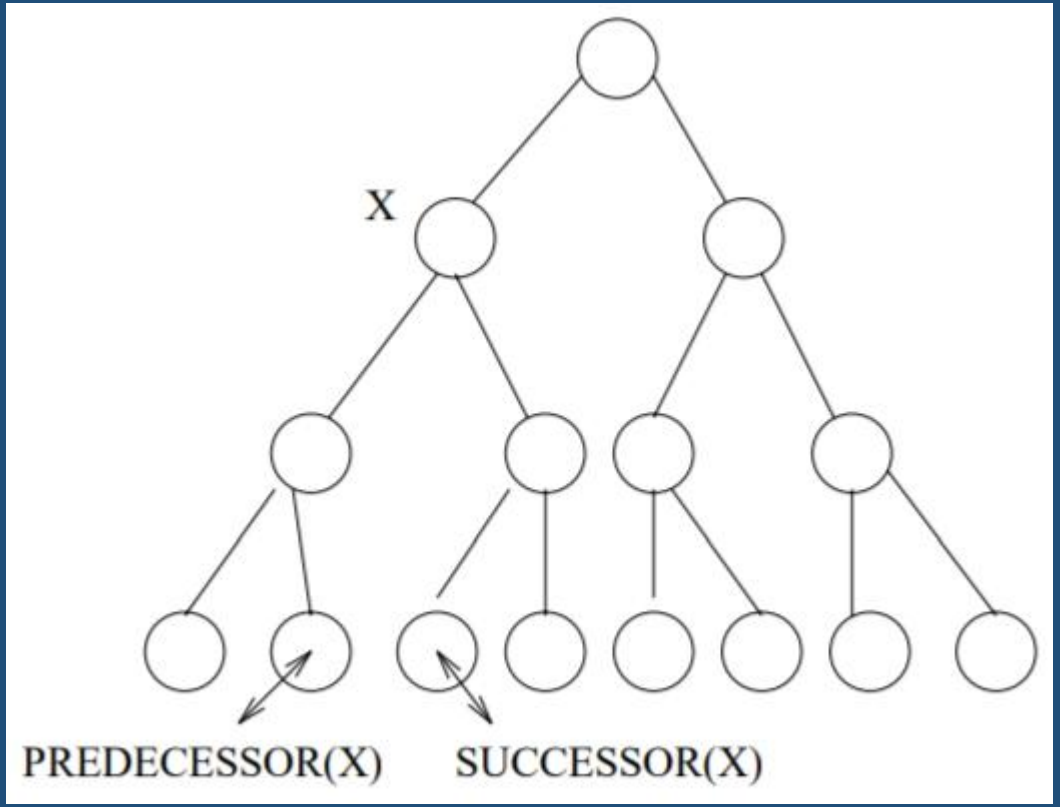
```
struct node* minimum(struct node* node)
{
    while (node->left != NULL)
        node = node->left;
    return node;
}

struct node* maximum(struct node* node)
{
    while (node->right != NULL)
        node = node->right;
    return node;
}
```



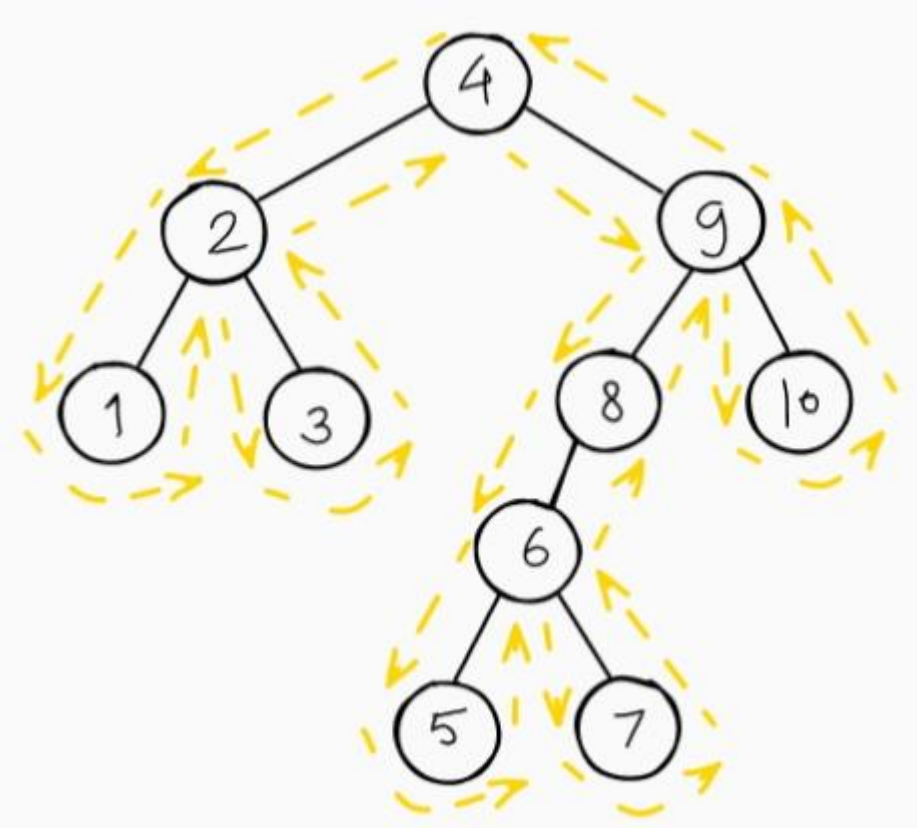


Predecessor and Successor

- If X has two children its predecessor is the maximum value in its left subtree and its successor the minimum value in its right subtree



Find predecessor and successor of X



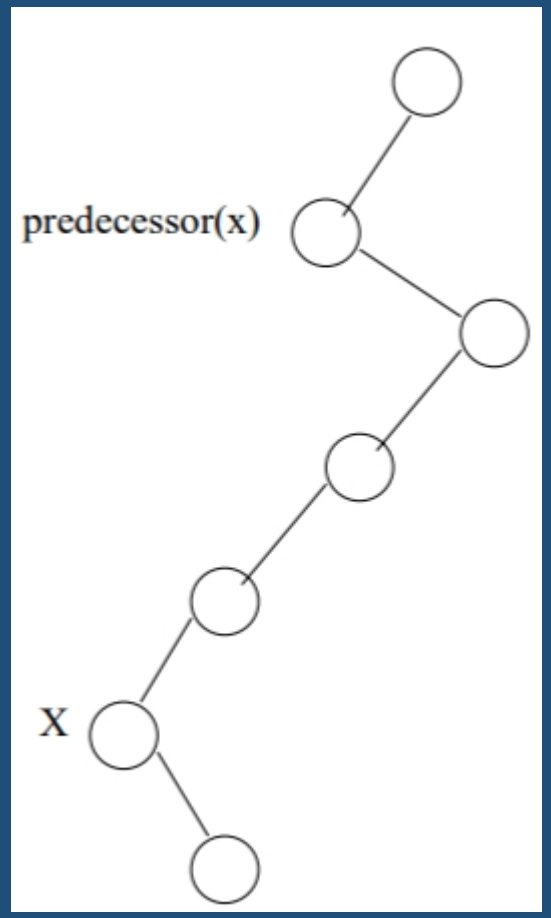
in-order - left, node, right

process a node when the dotten line passes by the bottom of a node

1 2 3 4 5 6 7 9 10

Successor, Predecessor and Inorder

- If it does not have a left child a nodes predecessor is its first left ancestor.
- If it does not have a right child a nodes successor is its first right ancestor




```
      8
    / \
   3   10
  / \    \
 1   6    14
    / \  /
   4   7 13
```

Predecessor = Maximum of the left subtree

```
struct node* predecessor(struct node* node)
{
    if (node->left != NULL)
        return maximum(node->left);
    struct node* ancestor = node->parent;
    while ((ancestor != NULL) && (node == ancestor->left))
    {
        node = ancestor;
        ancestor = node->parent;
    }
    return ancestor;
}
```

Successor = Minimum of the right subtree

```
struct node* successor(struct node* node)
{
    if (node->right != NULL)
        return minimum(node->right);
    struct node* ancestor = node->parent;
    while ((ancestor != NULL) && (node == ancestor->right))
    {
        node = ancestor;
        ancestor = node->parent;
    }
    return ancestor;
}
```

Complexity of operations

Operations	Complexity
insertion()	*O(h)*
search()	*O(h)*
minimum(), maximum()	*O(h)*
predecessor()	*O(h)*
successor()	*O(h)*
	Remark*: h* is the height of a binary tree. - At worst, *h* can be n-1. - At best, *h* can be log(n+1)−1.

Problems on binary search trees

(in grader1.cs.sci.ku.ac.th)

- [BSTs] Binary Search Tree Traversals
- [BSTs] Dynamic BST